

SCR2043 OPERATING SYSTEMS

Name : Iman Abadi Bin Mohd Nizwan
Student ID : A23CS0084
Section : 02

Marks

This lab assessment is designed to test your understanding and skills on some basic concepts and tools related to process monitoring and management in operating system. Please follow the instructions carefully and submit your answers in this word document and rename the file as **os-lab-assessment02-studentname-matricno.docx**.

Essential Steps Before Starting Lab Assessment 2:

1. Download necessary source codes:

Use the `wget` command to retrieve the following source code files to your Linux (or WSL or MacOS) environment:

```
wget -O mainprocess.c https://rebrand.ly/mainprocess_c  
wget -O subprocess1.c https://rebrand.ly/subprocess1_c  
wget -O subprocess2.c https://rebrand.ly/subprocess2_c
```

2. Compile the source files:

Use the `gcc` compiler to create executable files from the source code.

```
gcc mainprocess.c -o mainprocess  
gcc subprocess1.c -o subprocess1  
gcc subprocess2.c -o subprocess2
```

3. Execute the dummy processes:

Run all the dummy processes

```
./mainprocess &
```

Press **enter** two times.

4. The dummy processes are running for 2 hours. If you took longer than 2 hours on questions 1-9, please restart the main process with `./mainprocess &`.

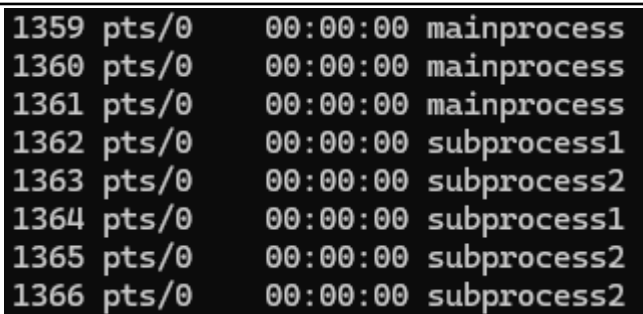
Lab Assessment 2 : Linux Process Monitoring and Management

Instructions:

1. Carefully execute each command as instructed in the questions.
2. Write down the exact command used for each task.
3. Capture a screenshot of the command's output.

Question 1

Use the `ps` command with the appropriate option to display a complete list of all running processes within the Linux operating system.

Command			
<code>ps -e</code>			
Output			
			

Question 2

Employ the `ps` command with necessary options to unveil comprehensive details about each running process.

Command
<code>ps -aux</code>
Output

iman	1147	0.0	0.7	13756	7300	?	S	04:41	0:01	sshd: iman@pts/0
iman	1148	0.0	0.5	5644	4992	pts/0	Ss	04:41	0:00	-bash
iman	1359	0.0	0.1	2680	1536	pts/0	S	04:43	0:00	./mainprocess
iman	1360	0.0	0.0	2680	128	pts/0	S	04:43	0:00	./mainprocess
iman	1361	0.0	0.0	2680	128	pts/0	S	04:43	0:00	./mainprocess
iman	1362	0.0	0.1	2680	1536	pts/0	S	04:43	0:00	./subprocess1
iman	1363	0.0	0.1	2680	1536	pts/0	S	04:43	0:00	./subprocess2
iman	1364	0.0	0.1	2680	1536	pts/0	S	04:43	0:00	./subprocess1
iman	1365	0.0	0.1	2680	1536	pts/0	S	04:43	0:00	./subprocess2
iman	1366	0.0	0.1	2680	1536	pts/0	S	04:43	0:00	./subprocess2
root	1376	0.0	0.0	0	0	?	I	04:50	0:00	[kworker/u5:2-events_unbound]
root	1379	0.1	0.0	0	0	?	I	04:50	0:01	[kworker/1:0-events]
root	1389	0.0	0.0	0	0	?	I	04:56	0:00	[kworker/u6:0-events_power_eff]
root	1398	0.4	0.0	0	0	?	I	05:00	0:02	[kworker/0:0-mpt_poll_0]
root	1399	0.0	0.0	0	0	?	I	05:00	0:00	[kworker/u5:1-events_unbound]
root	1403	0.0	0.0	0	0	?	I	05:00	0:00	[kworker/1:3-cgroup_destroy]
root	1404	0.0	0.0	0	0	?	I	05:00	0:00	[kworker/u6:3-events_unbound]
root	1426	0.0	0.0	0	0	?	I	05:10	0:00	[kworker/1:1-cgroup_destroy]
iman	1427	0.0	0.4	9192	4736	pts/0	R+	05:10	0:00	ps -aux

Question 3

Use the `ps` command with some tools to only list processes named "subprocess" and show some info about them.

Command										
ps aux grep -i "[s]ubprocess"										
Output										
iman	1362	0.0	0.1	2680	1536	pts/0	S	04:43	0:00	./subprocess1
iman	1363	0.0	0.1	2680	1536	pts/0	S	04:43	0:00	./subprocess2
iman	1364	0.0	0.1	2680	1536	pts/0	S	04:43	0:00	./subprocess1
iman	1365	0.0	0.1	2680	1536	pts/0	S	04:43	0:00	./subprocess2
iman	1366	0.0	0.1	2680	1536	pts/0	S	04:43	0:00	./subprocess2

Question 4

Execute the `ps` command, specifying options that reveal only the following columns:

- Process ID (pid)
- Owner of the process (user)
- CPU percentage (pcpu)
- Memory percentage (pmem)
- Command (cmd)

Command				
ps -eo pid,user,pcpu,pmem,cmd grep -i "[s]ubprocess"				
Output				
1362	iman	0.0	0.1	./subprocess1
1363	iman	0.0	0.1	./subprocess2
1364	iman	0.0	0.1	./subprocess1
1365	iman	0.0	0.1	./subprocess2
1366	iman	0.0	0.1	./subprocess2

Question 5

Building on the `ps` command used in Question 4, can you add an option to sort the listed processes by their memory usage (`pmem`)?

Command	
<pre>ps -eo pid,user,pcpu,pmem,cmd --sort=-pmem grep -i "[s]ubprocess"</pre>	
Output	
<pre>1362 iman 0.0 0.1 ./subprocess1 1363 iman 0.0 0.1 ./subprocess2 1364 iman 0.0 0.1 ./subprocess1 1365 iman 0.0 0.1 ./subprocess2 1366 iman 0.0 0.1 ./subprocess2</pre>	

Question 6

Construct a command using `ps`, suitable options, and any additional tools to visualize the hierarchical structure (tree-like) of the following processes:

- "mainprocess"
- "subprocess1"
- "subprocess2"

Command	
<pre>ps -f --forest -C mainprocess,subprocess1,subprocess2</pre>	
Output	
<pre>UID PID PPID C STIME TTY TIME CMD iman 858 836 0 08:37 pts/0 00:00:00 ./mainprocess iman 859 858 0 08:37 pts/0 00:00:00 _ ./mainprocess iman 864 859 0 08:37 pts/0 00:00:00 _ ./subprocess1 iman 865 859 0 08:37 pts/0 00:00:00 _ ./subprocess1 iman 860 858 0 08:37 pts/0 00:00:00 _ _ ./mainprocess iman 861 860 0 08:37 pts/0 00:00:00 _ ./subprocess2 iman 862 860 0 08:37 pts/0 00:00:00 _ ./subprocess2 iman 863 860 0 08:37 pts/0 00:00:00 _ ./subprocess2</pre>	

Question 7

Use `pstree` command with option that show the number of threads to each process.

Command	
<pre>pstree -p -T grep -E 'mainprocess subprocess1 subprocess2'</pre>	

Question 10

Write a short C or Python code (choose only one language) demonstrating multiprocessing with `fork()` and `wait()`. Compile and/or run the code. Show the output.

Source Code:

Python Source Code

```
import os
import time

pid = os.fork()
if pid == 0:
    print(f'Child (PID={os.getpid()}) started. Parent PID={os.getppid()}')
    time.sleep(2)
    print(f'Child (PID={os.getpid()}) finished.')
    os._exit(42)
else:
    print(f'Parent (PID={os.getpid()}) waiting for child (PID={pid})...')
    _, status = os.wait()
    print(f'Parent: Child exited with status {status >> 8}')
```

Output:

```
iman@secre2043:~$ python3 fork_wait.py
Parent (PID=1202) waiting for child (PID=1203)...
Child (PID=1203) started. Parent PID=1202
Child (PID=1203) finished.
Parent: Child exited with status 42
```